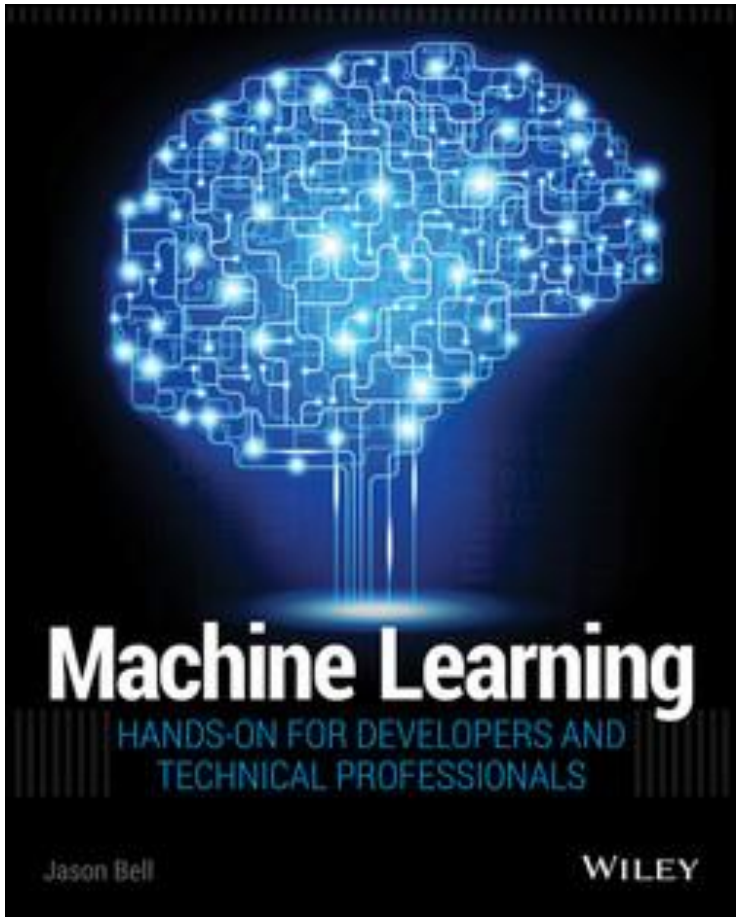
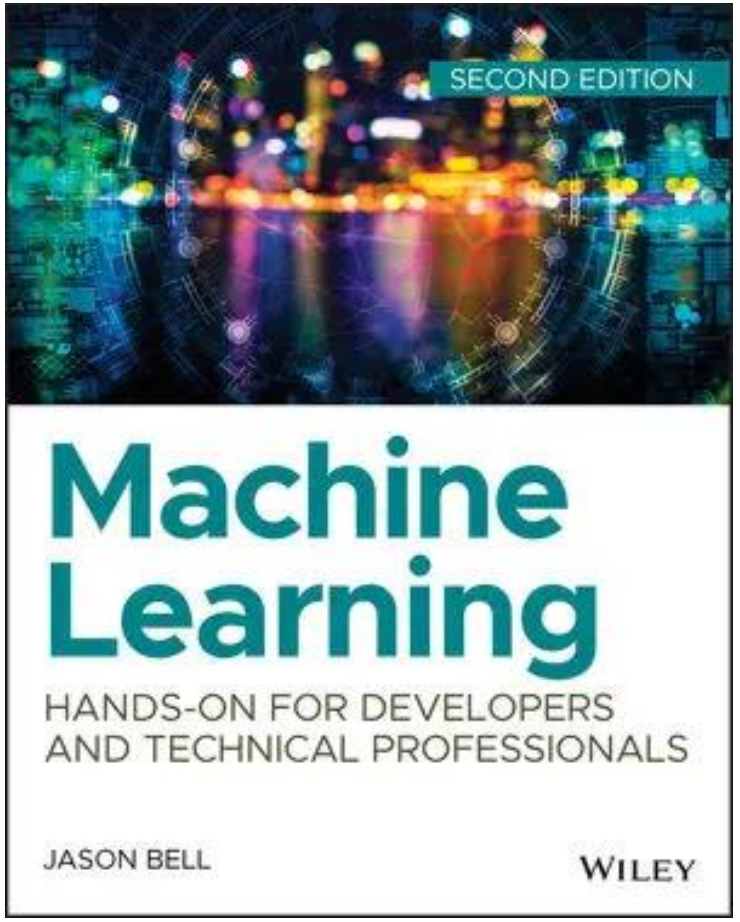
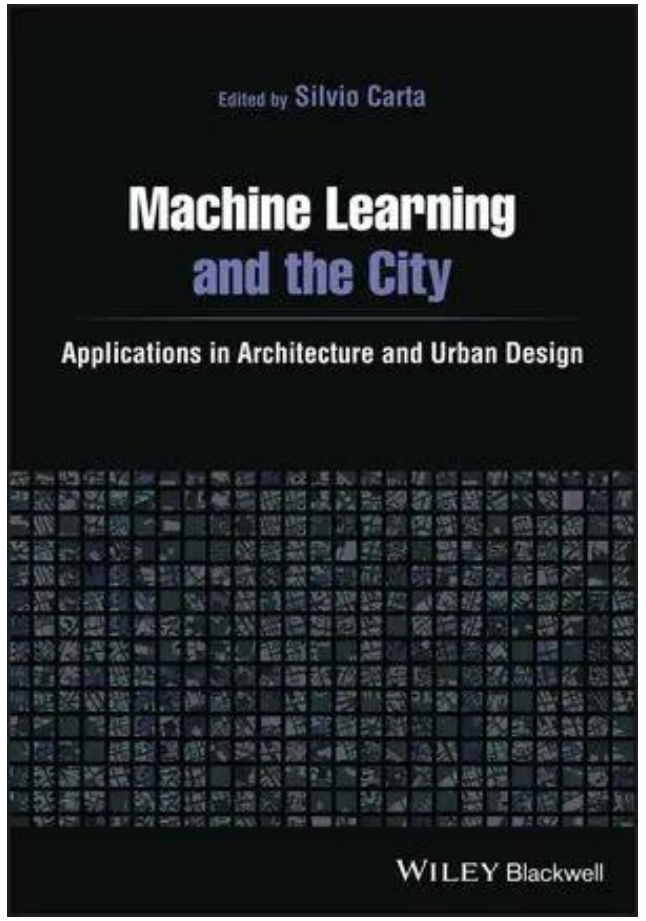
WHAT THEY DON'T TELL YOU IN THE CLAUDE DEMO

Why did he get invited?

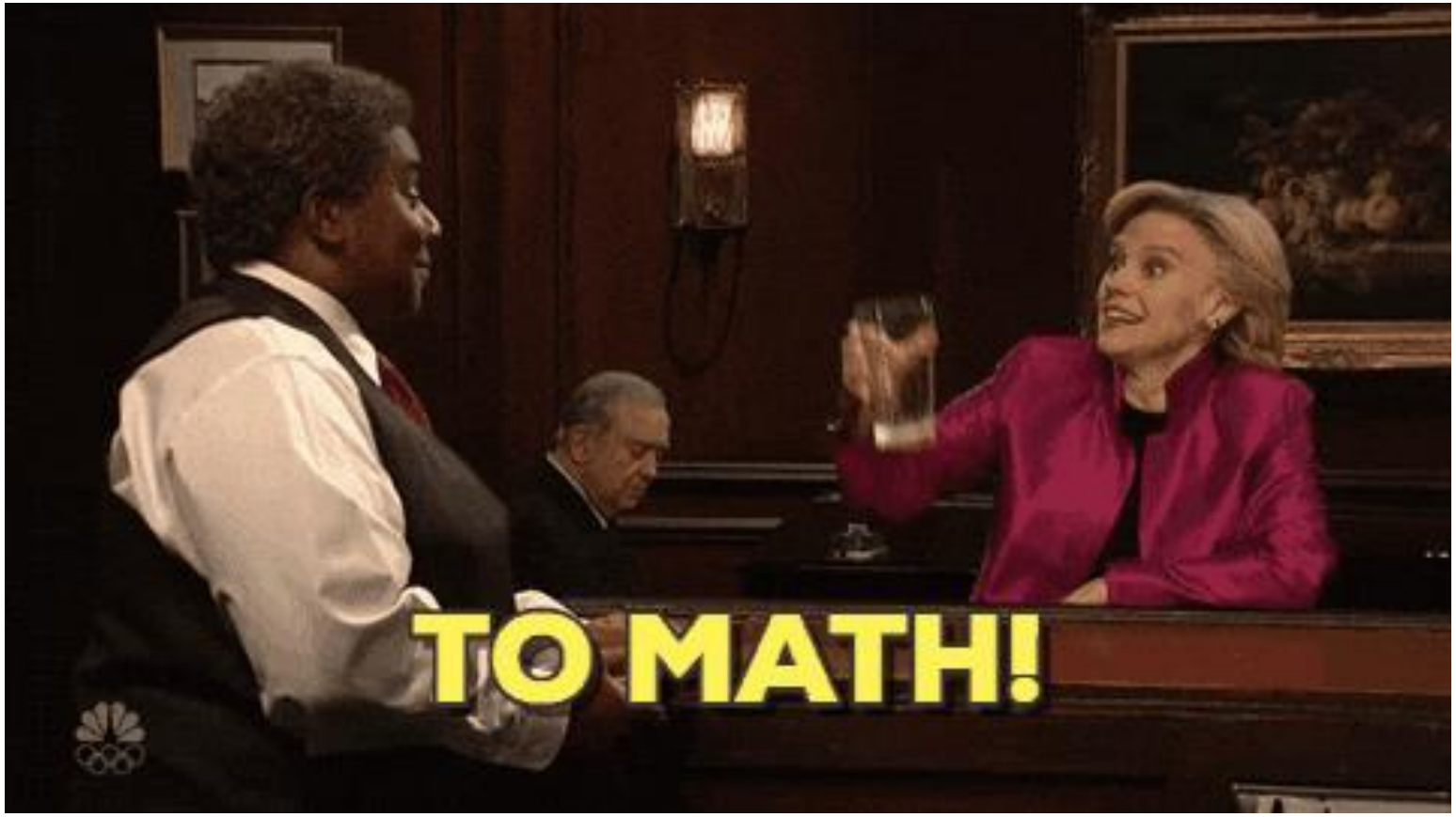




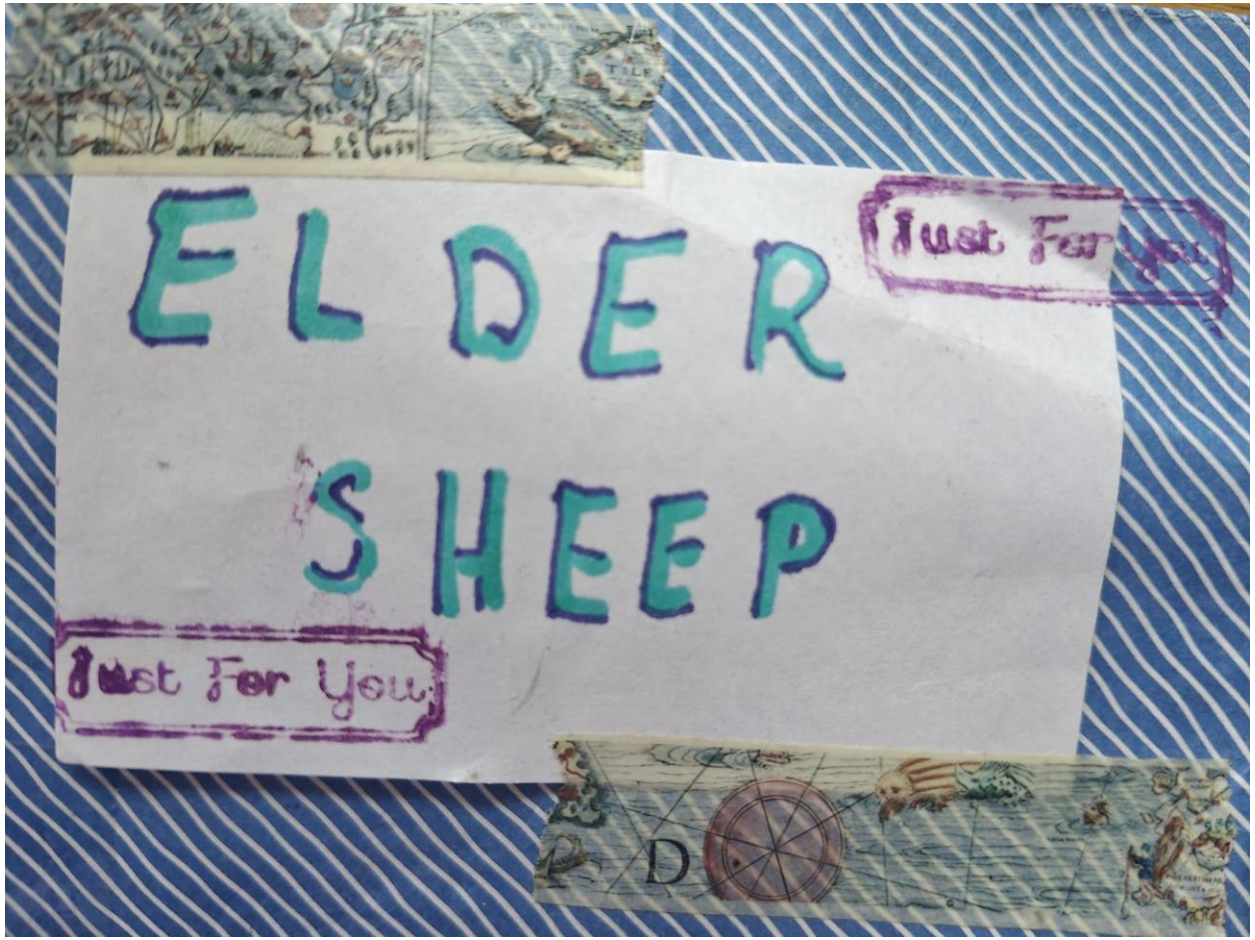


Why this buffoon?

It’s not negativity, it’s realistic appraisal.



Elder’s have access to history.



QUESTION…?

THE QUESTION NOBODY ASKS IN THE DEMO

Every demo shows you what the tool can do.

None of them ask what you *stop doing* once you rely on it.

What I'll Cover

- The lie the demo tells you
- What it costs you to outsource the thinking
- The stakes: data, accountability, dependency
- A playbook you can use tomorrow
- What to take back to your team

This replaced you…

$$P(x_{t+1} \mid x_1, \ldots, x_t) = \textbf{softmax}(\, W_U \, h_t \,)$$

- h_t: the context, as one vector
- W_U: scores every word
- **softmax**: scores → probabilities

Where h_t comes from

$$h_t = \text{Transformer}(x_1, \ldots, x_t)$$

$$\textbf{softmax}(QK^T / \sqrt{d_k})\, V$$

Attention lets each word look back at earlier words.

How it learns

$$L = -\Sigma_t \log P_\theta(x_t \mid x_{<t})$$

Nudge the weights so the real next word gets a higher probability.

This replaced you…

```
def add_user_message(messages, message):
    user_message = {
        "role": "user",
        "content": message.content if hasattr(message, "content") else message,
    }
    messages.append(user_message)


def add_assistant_message(messages, message):
    assistant_message = {
        "role": "assistant",
        "content": message.content if hasattr(message, "content") else message,
    }
    messages.append(assistant_message)


def chat(messages, system=None, temperature=1.0, stop_sequences=[], tools=None):
    params = {
        "model": model,
        "max_tokens": 1000,
        "messages": messages,
        "temperature": temperature,
        "stop_sequences": stop_sequences,
    }
    if system:
        params["system"] = system
    if tools:
        params["tools"] = tools

    return client.messages.create(**params)
```



The Demo Lie

- Prompt -> output -> applause!
- The thinking happened somewhere you can't see
- Nobody asked: did the person understand what they shipped?

Outsourcing the Thinking

- Writing code is how you think about the problem - review replaces understanding with plausibility-checking
- “It works” is not the same as “I know why it works”
- If you can't write it, you can't review it - “AI operators” stall when the tool's down or the problem is novel
- Single-vendor risk: pricing, model changes, outages, terms

One Diff, One Afternoon

The Prompt

Add a redeem(code, user_id, price_cents) function for our launch coupons.

Each code can be used once, by one user.

Codes expire.

Return the discounted price in cents.

Raise clear errors for unknown, expired, or already-used codes.

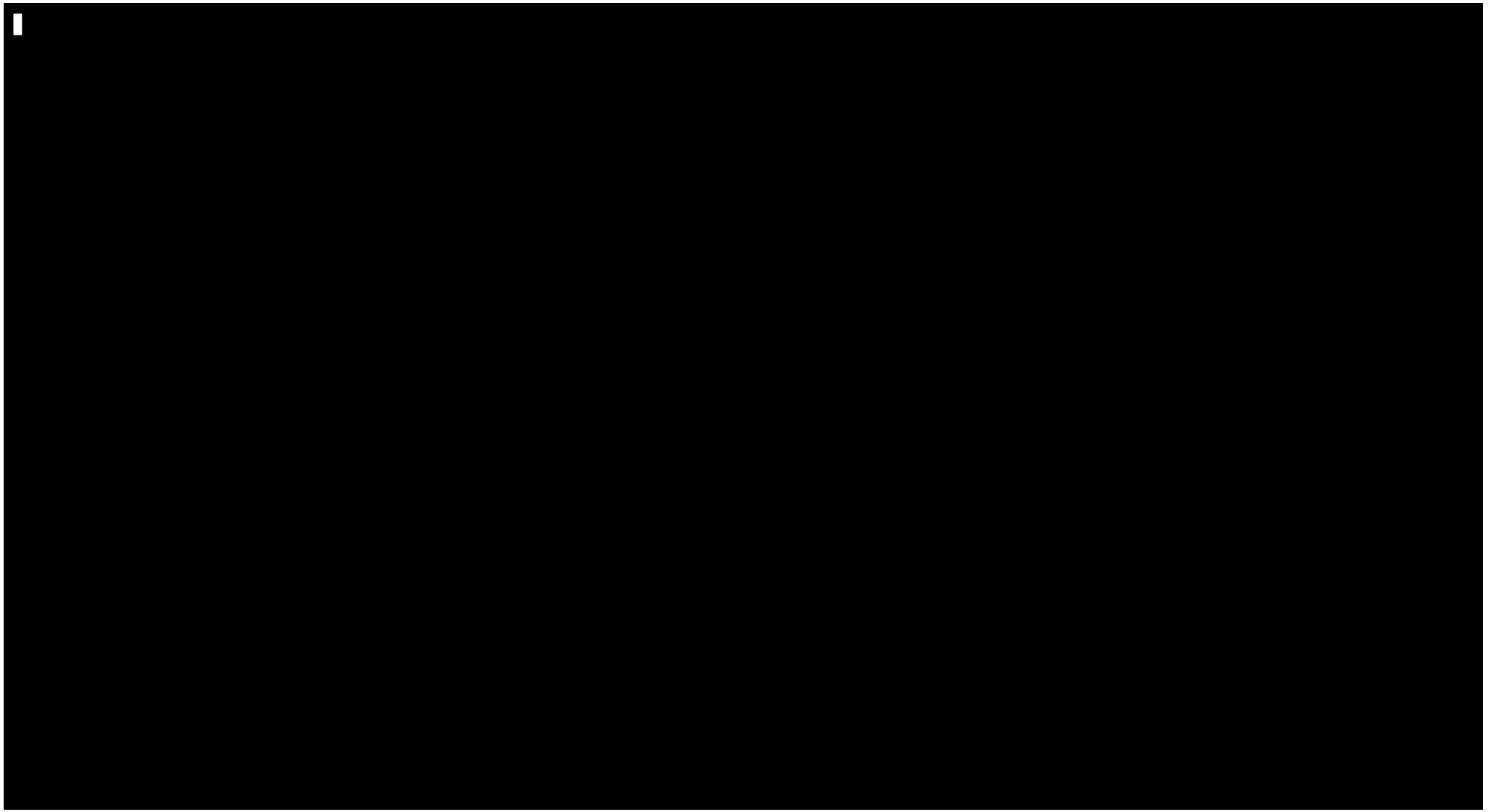
Add tests.

One Diff, One Afternoon

What Got Shipped…
A clean `redeem()` function with custom exceptions, correct rounding, and 6 readable tests covering every line of the spec.

It is green and looks fine in review.

One Diff, One Afternoon



One Diff, One Afternoon

What actually broke?
redeem() does a read, a check, then a write, and the lock only protects each call separately.

When 50 people hit “Apply” together, they all read redeemed_by = None before anyone writes.

All 50 get the discount, and it surfaces on launch day, not in CI.

One Diff, One Afternoon

The Tell

One review question: “**What happens if two requests hit this at the same time?**”

The specification said “used once”, but none of the tests covers concurrency.

Review should also ask for an atomic operation (a conditional `UPDATE ... WHERE redeemed_by IS NULL`, or a unique constraint) instead of check-then-act.

One Diff, One Afternoon



One Diff, One Afternoon

The code reads correctly and every test passes.

The gap is in what the tests never ask.

Claude wrote tests for the spec you gave it, so they could not catch a requirement nobody stated, and the reviewer saw green and moved on.

One Diff, One Afternoon

“The quality of the question determines the quality of the answer.”: Robert Fripp

The Stakes

- The tool has no liability. You do.
- Prompts are data leaving your estate - treat them like a leak until you know your retention settings
- Security review: apply contractor-level paranoia
- “Explain it back” as the accountability rule

The Playbook: Principles

- Think first, prompt second - write the design and tests before you generate
- Use it for boilerplate and scaffolding, not business logic you haven't thought through
- Keep manual practice alive: some work done without the tool

The Playbook: Practices

- Diffs small enough to review in one sitting
- Explain it back - out loud, before it merges
- Data red lines: write down what may never go into a prompt
- Review it as if a stranger wrote it

If You Run a Team

- Make “explain it back” a review gate, not a personal habit
- Put data red lines in writing - policy, not tribal knowledge
- An engineer who stalls without the tool has a skill gap, not a preference
- Single-vendor exposure is a procurement question, not just a dev one

IS IT OVER NOW?

THE QUESTION THAT MATTERS

Stop asking “look what I *built*” and start asking “look what I *understand*, and who has my data.”

The demo shows you speed. The post-mortem shows you the bill.

Senior engineering is knowing both before you swipe the card.

JASE'S VERSION

Questions & Answers

Github: https://github.com/jasebell/techtides2026

Want to talk, connect with me: https://linkedin.com/in/jasonbelldata

The Playlist: open.spotify.com/playlist/22jzEBG4yvo0N60yVHosT0